

Safe-Escape MPPI: Coordinating Online Local-Minima Escape with Control-Barrier-Function Safety in a Nav2-Native Controller

Jung Mo Kang

Abstract—The Model Predictive Path Integral (MPPI) controller shipped with ROS 2 Nav2 is a widely used deployable local controller for mobile robots, yet in narrow, crowded, dynamic spaces it can stall in local minima, and its critics avoid collisions only through soft costs, with no formal safety guarantee. Methods exist for each weakness, and several works outside Nav2 address both together, but we know of no public Nav2 controller plugin that combines local-minima escape with a barrier-based safety filter. This system paper presents Safe-Escape MPPI (SE-MPPI), a Nav2 controller plugin that integrates online local-minima detection-and-escape for MPPI with a control-barrier-function (CBF) safety filter for tracked dynamic obstacles. One path-progress entrapment signal switches repulsion and free-space gap costs into the MPPI rollouts and, through a coordinator, raises the barrier’s class- K gain. By a standard argument for time-varying CBF gains, this schedule preserves forward invariance of the barrier’s safe set, a conditional guarantee that covers only a look-ahead point and the tracked dynamic obstacles, assumes that the barrier inequality holds between control updates, and requires the quadratic program to use no slack, a condition that the benchmark QP violates at least once in most trials with moving obstacles. In a randomized, paired 1,200-trial benchmark run on a Python 2D mirror of the controller’s primitives, not on the Nav2 controller itself, the detection-and-escape layer is the decisive contribution on both trap families, with the free-space gap search load-bearing. The escape-safety coordination instead matches an independent escape+CBF stack – a null result: outcomes are insensitive to the barrier-gain schedule even where the barrier engages – so we claim the coordination only as a mechanism, not as an empirical gain. The controller also runs inside a live ROS 2 Jazzy + Nav2 + Gazebo stack, and we report the deployment faults live integration exposes, although no live run has yet reached its goal.

Index Terms—Model predictive control, control barrier functions, collision avoidance, local-minima escape, motion and path planning, autonomous navigation, ROS 2 Nav2.

I. Introduction

Autonomous mobile robots increasingly operate in cluttered, dynamic indoor environments – warehouses, hospitals, service floors – where the local controller must be simultaneously fast, non-conservative, and safe. Within the ROS 2 Nav2 ecosystem, the MPPI controller (`nav2_mppi_controller`) [1] is a widely used high-performance option: it supports differential, omnidirec-

tional, and Ackermann robots and runs at 50+ Hz on CPU by sampling batch_size rollouts, scoring them with a bank of critics, and combining them with an information-theoretic softmax [2].

Two weaknesses, however, recur exactly in the environments that matter most:

- 1) **Local-minima entrapment.** A finite prediction horizon (by default $56 \times 0.05\text{s} \approx 2.8\text{s}$) and zero-mean Gaussian sampling cause the robot to stall or oscillate in non-convex traps. Within the MPPI critic set, Nav2’s mitigations (`PreferForwardCritic`, `TwirlingCritic`) are always-on cost shaping that neither detects entrapment nor steers an escape. At the stack level, Nav2 does detect a stall: the controller server’s progress checker reports a failure when the robot has not moved a set distance within a set time, and the default behavior tree then clears the costmaps and runs generic recoveries such as spinning, waiting, and backing up [1], [3], [4]. These recoveries are not directed toward an opening. In the MPPI literature, escape methods either reshape the sampling distribution all the time [5], [6], [7], [8], [9] or add guidance: GP-MPPI recommends a free-space subgoal from a learned navigability surface [10], and repulsive-potential augmentation adds an artificial potential field to the MPPI cost [11], which DRPA-MPPI switches on only when it detects entrapment [12]. None of these carries a safety certificate.
- 2) **No formal safety.** In Nav2 MPPI, avoidance is encoded as soft costs (`ObstaclesCritic`, `CostCritic`); there is no forward-invariance guarantee, and crucially no prediction of dynamic-obstacle motion – the costmap is a present-time snapshot and `collision_monitor` is purely reactive. CBF-MPPI fusions supply safety [13], [14], [15], [16], [17], but none of them detects entrapment or adds an escape mechanism (GS-MPPI counters the myopic behavior often associated with CBFs through long-term performance terms), and none ships as a Nav2 controller. Among CBF integrations with Nav2 that we could verify, the Nav2 documentation describes inserting a third-party product, the 3Laws Supervisor, which its vendor describes as based on CBFs, into the

command chain after the controller [18], [19]; it is a separate node rather than a controller plugin, and neither source describes a mechanism for escaping local minima.

These weaknesses also interact: being conservative for safety invites entrapment, and escaping aggressively invites collision. Several works outside Nav2 and MPPI address both at once, among them model predictive control that escapes local minima while keeping a safe backup trajectory [20], gap-based local planners with provable collision-free properties [21], [22], [23], and modulated CBF filters [24] that reduce or eliminate, among moving obstacles, the spurious equilibria that CBF quadratic programs (QPs) can introduce [25], with concurrent extensions [26], [27]. Fig. 2(a) (Sec. VI-A) illustrates the first weakness and SE-MPPI's response in the standalone 2D mirror of the controller's primitives: stock MPPI stalls before a U-shaped trap while SE-MPPI detects the entrapment and rounds it. In SE-MPPI, the entrapment detector is driven by monotone progress along the global path and runs inside the controller rather than at the stack level. This is a system contribution, and we state it as such.

Contributions.

- C1 (system). A public Nav2 local-controller plugin that combines online local-minima detection-and-escape with a CBF safety filter for tracked dynamic obstacles, packaged as a `nav2_core::Controller` plus a pluginlib MPPI critic. Each ingredient has precedent: gated repulsion in DRPA-MPPI [12], free-space subgoal and gap guidance in GP-MPPI [10] and gap-based planners [21], [22], CBF filtering of MPPI commands [15], and a supervisor for Nav2 that its vendor describes as CBF-based [18], [19]. What is new is their integration inside one Nav2 controller; to our knowledge no public controller plugin already provides it.
- C2 (algorithm). An entrapment-gated gain schedule: a rule that raises the CBF class- $\mathcal{K}$ gain on detected entrapment and drops it back on an imminent time-to-collision (TTC). Online adaptation of CBF gains is established, including selecting class- $\mathcal{K}$ parameters to reduce predicted deadlock [28] and letting MPPI optimize a barrier-rate parameter [14]; our rule is a simpler, rule-based two-level instance of online gain adaptation whose specific elements are its trigger and its TTC override. Its invariance property (Sec. IV-E) is a known result for time-varying class- $\mathcal{K}$ terms [29], [30], which we restate as a conditional lemma: it covers the look-ahead point and the tracked dynamic obstacles, assumes the barrier inequality holds between samples, and requires a slack-free QP. In our 2D benchmark the schedule is empirically indistinguishable from an independent escape+CBF stack (Sec. VI-C1).
- C3 (dynamic safety). Integration of a look-ahead-point CBF safety filter for a differential-drive robot,

fed by a costmap-based dynamic-obstacle tracker, as a QP at the controller's output.

- C4 (reproducibility). An open Apache-2.0 implementation,¹ a standalone 2D mechanism validation, a randomized 1,200-trial benchmark on a Python 2D mirror of the controller's primitives (200 seeded worlds, each run under six configurations) with paired significance tests, and a specified full-scale ROS 2 physics-benchmark protocol.

We are deliberately conservative about novelty and about what we claim empirically. Fusing CBFs with MPPI is not new, and neither is gated escape, online adaptation of the CBF gain, or treating escape and safety together; the paper is best read as a system report on putting these pieces into one Nav2 controller. The measured win, in the 2D mirror, is the online detection-and-escape layer ($0\% \rightarrow 88\text{--}90\%$ on the static U-trap family, Holm-adjusted $p = 1.4 \times 10^{-9}$, and $0\% \rightarrow 62\text{--}78\%$ on the narrow-dynamic family, $p_{\text{adj}} = 1.1 \times 10^{-6}$, with the gap search load-bearing; Sec. VI-C). In the mirror the gap acts as a temporary subgoal, close to the subgoal guidance of GP-MPPI [10]; the gap cost of the C++ critic has not been ablated. The escape-safety coordination (C2) has the conditional invariance property of Sec. IV-E yet, in the regime we could test, is empirically indistinguishable from an independent escape+CBF stack; we report this null openly and delimit where coordination should matter. The remaining claims are Nav2-native deployment and the deployment faults a live stack exposes; no live run has yet reached its goal, and no stack-level or third-party baseline has been run (Sec. VII).

II. Related Work

MPPI and local controllers. Nav2 MPPI [1], [2], TEb [31], DWB (a Nav2 DWA-family planner) [32], and Regulated Pure Pursuit [33] are the standard deployable local planners. MPPI's critic-plugin interface (`mppi::critics::CriticFunction`) exposes per-rollout trajectories, the reference path, the goal, collision flags, and the furthest-reached path index – i.e., the signals needed for entrapment detection are already available, and a custom critic can be inserted without forking the optimizer. Around the controller, Nav2 already handles stalls at the stack level: the controller server runs a progress checker, by default `SimpleProgressChecker`, which reports a failure when the robot has not moved a set radius within a set time allowance, and the default behavior tree answers with costmap clearing, spinning, waiting, and backing up [3], [4]. SE-MPPI's detector differs in that it runs inside the controller on progress along the path and steers the escape instead of triggering generic recoveries; the two mechanisms can coexist.

Local-minima escape. Most escape methods for MPPI are always-on changes of the sampling distribution: log-

¹Code, benchmark scripts, and raw trial data: <https://github.com/kjungmo/se-mppi-nav2> (Apache-2.0); see the Appendix for the regeneration commands.

MPPI [5], Tsallis-MPPI [6], SVG-MPPI (Stein mode-seeking) [7], Biased-MPPI [8], and learned sampling distributions for MPC (Sacks and Boots) [9]. Others add guidance. GP-MPPI learns a sparse-Gaussian-process model of the navigable space around the robot and recommends a free-space subgoal to the local MPPI planner, which lets the robot escape local minima without a global map [10]. Fuke et al. add a repulsive artificial potential to MPPI to avoid local minima without global path guidance [11], and their DRPA-MPPI detects potential entrapment on the predicted trajectories and only then switches to a cost with repulsion away from the local minimum [12]. DRPA-MPPI is the closest detect-and-switch method and GP-MPPI the closest subgoal method; neither has a CBF or other formal safety layer, and neither is Nav2-integrated. SE-MPPI combines both ideas in a simpler form, a path-progress stall detector gating a classical repulsive potential [34] and a ray-cast gap bearing, and its safety filter runs alongside the escape rather than gating it. Gap-based local planners are closer still to the overall architecture. Potential Gap combines gap-based local navigation with artificial potential fields and restores provable collision-free properties for realistic robot models [21]; Safer Gap adds nonlinear model predictive control with keyhole zeroing-barrier-function constraints and a point-wise optimization-based safety filter as the final layer [22]; and Dynamic Gap extends provably collision-free gap-based navigation to dynamic environments by modeling how gaps evolve over time [23]. Relative to this line, SE-MPPI uses a gap only as a gated escape heuristic inside MPPI, and its barrier covers only tracked dynamic obstacles.

Escape and safety together. Several works treat entrapment and safety jointly. Soloperto et al. optimize an exploration trajectory together with a safe backup trajectory in a model predictive controller and prove convergence, feasibility, and constraint satisfaction under partially unknown constraints [20]. The modulated CBF-QP of Xue and Figueroa reduces or eliminates, in dynamic environments, the spurious equilibria that CBF-QP filters create [25], and was tested on Ridgeback and Fetch robots [24]. Concurrent work adds learned predictive barriers for moving concave obstacles to this filter [26], and pairs a mixed-integer MPC planner with a Minkowski-difference CBF filter to mitigate the local-minimum behavior of reactive CBF navigation in U-shaped and maze-like environments [27]. These methods mitigate or avoid minima inside the planner or the barrier itself, whereas SE-MPPI leaves the barrier unchanged and escapes through the MPPI cost. None of them is packaged as a Nav2 controller.

CBF $\times$ MPPI safety. Three fusion styles exist as research code: CBF conditions in the rollout cost or the sampling (Shield-MPPI [13], which adds a local repair step, and BR-MPPI [14], which imposes the CBF condition as an equality with a parametric linear class- $\mathcal{K}$ function, augments the state with its parameter, and lets MPPI optimize the parameter's rate as an extra input), an output CBF-QP filter on the MPPI command (SCBF-

MPPI [15], built on the CBFKit toolbox [35]), and per-rollout safety filtering that makes every sample provably safe (GS-MPPI [17], DualGuard [16], which also filters the final command). For dynamic obstacles, DPCBF [36] uses a parabolic safe set that is less conservative than collision-cone barriers but is QP-only and not embedded in MPPI. Our filter builds on the classical CBF-QP [37], [38]. None of these works ships as a Nav2 controller (CBFKit builds a standalone node), and none detects entrapment or adds an escape mechanism, although GS-MPPI counters the myopic behavior often associated with CBFs through long-term performance terms [17]. Among Nav2 integrations of a CBF that we could verify, the official documentation describes inserting the third-party 3Laws Supervisor, which its vendor describes as based on CBFs, between the velocity smoother and the collision monitor [18], [19]; it acts on the controller's output as a separate node, and neither source describes a local-minimum escape mechanism. SE-MPPI instead places its filter inside the controller and couples it to the escape layer through its gain.

Online adaptation of the CBF gain. Time-varying and adapted class- $\mathcal{K}$ terms are well studied. Adaptive CBFs multiply the class- $\mathcal{K}$ functions by time-varying penalty functions and prove forward invariance of the safe set [29]; optimal-decay CBF-QPs optimize the decay rate point-wise in time to keep the QP feasible [39]; and rate-tunable CBFs adapt the class- $\mathcal{K}$ functions online and derive conditions under which multiple constraints can share the control input [30]. Closest to our trigger, Kim, Kee, and Panagou adapt the parameters of input-constrained CBFs online with a learned ensemble model and select, among the candidates that pass their uncertainty and risk checks, the class- $\mathcal{K}$ parameters with the minimum predicted deadlock time [28]; BR-MPPI lets MPPI optimize the class- $\mathcal{K}$ parameter directly [14]. Concurrent work switches nominal objectives when CBF and control-Lyapunov constraints conflict, to reduce slowdowns and deadlocks [40], and schedules the gain of a frontier barrier on an occupancy grid by how well the map is explored [41]. SE-MPPI's coordinator is a rule-based, two-level instance of this idea, triggered by the entrapment flag and overridden by time-to-collision, and its invariance property (Sec. IV-E) follows the same argument as the relative-degree-one case of the adaptive-CBF result [29, Thm. 3], there stated for Lipschitz controllers.

Conformal prediction for safe control (background, not claimed). The released controller carries an online conformal margin that inflates the CBF effective radius under prediction uncertainty; this is an implementation detail deferred to follow-up work (Sec. VII, SE-Predict) and is not a contribution of this paper. That margin draws on the established line of conformal-prediction-for-safe-control work [42], [43], [44], [45], [46]; Paper 1's claims are C1–C4 and stand without it.

Positioning. Table I places the reviewed systems against four factors: (a) shipping as a Nav2 controller plugin, (b) online local-minimum escape or avoidance, (c) an

explicit run-time safety constraint, filter or backup, and (d) online adaptation of the class- $\mathcal{K}$ gain. No reviewed system holds all four, but several hold two, fully or in part: stock Nav2 pairs its controller plugins with stack-level stall recovery, gap-based planners, backup-trajectory MPC and modulated CBFs combine local-minimum handling with a safety layer, and adaptive-gain CBFs and BR-MPPI pair a barrier with gain adaptation, with deadlock-aware adaptation also holding the second factor in part. The delta of SE-MPPI is therefore the conjunction inside one Nav2 controller plugin, not any single element. Its own marks are not all full: its barrier covers only tracked dynamic obstacles, and its escape is demonstrated only in the Python 2D mirror. Its gain rule differs from prior gain adaptation mainly in its trigger, and, as Sec. VI-C1 shows, we could not measure an outcome benefit from it.

III. Problem Formulation

Consider a differential-drive robot with pose (x, y, θ) and control $u = (v, \omega)$, tracking a global path σ produced by a Nav2 global planner, amid static structure and a set $\mathcal{O}$ of dynamic obstacles o with position p_o , radius R_o , and (estimated) velocity v_o . Let r be the robot's inscribed radius and m a safety margin. We want a control policy that (G1) makes monotone progress along σ – i.e., does not stall in a local minimum – and (G2) keeps the robot clear of the obstacles in $\mathcal{O}$ through a barrier whose forward invariance holds under stated conditions, while (G3) running as a Nav2 controller at ≥ 10 Hz on CPU.

The tension: G1 may require maneuvers that approach obstacles (to round a trap), whereas a naive safety filter for G2 forbids exactly those maneuvers. SE-MPPI's coordinator (Sec. IV-E) couples the escape layer for G1 and the barrier for G2 through the barrier gain.

IV. Method: SE-MPPI

A. Overview

SE-MPPI subclasses the stock MPPIController, reusing its optimizer for the nominal command, then post-processes that command each cycle:

- 1) Entrapment detection from monotone global-path progress (single source of truth, shared with the critic).
- 2) Escape at sampling time: an EscapeCritic injects distance-field APF and free-space gap-attraction costs only when entrapment is detected.
- 3) Dynamic-obstacle tracking from the local costmap.
- 4) Coordination: resolve the CBF gain α from entrapment and TTC.
- 5) CBF safety filter: project the nominal (v, ω) onto the CBF-safe set via a small QP.

The escape and safety layers thus act at complementary points – sampling-time cost shaping and output-time projection – unified by a shared entrapment signal and the coordinated gain.

Fig. 1 summarizes the resulting data flow.

B. Entrapment detection (single source of truth)

nearestPathIndex(σ) is non-monotone, so we track the furthest reached path index $k_{\max}$ and declare entrapment when $k_{\max}$ fails to advance for a window of W cycles (default 30; the 2D benchmark of Sec. VI-C uses 15). Entrapment is suppressed within a goal tolerance so a robot finishing at the path end does not trigger a false escape. Within SE-MPPI, the controller's detector is the single entrapment source; the EscapeCritic and the coordinator read the same shared atomic state via a registry keyed by the controller name, so escape and safety always agree on whether the robot is trapped.

C. Escape (detect-and-switch)

On entrapment the EscapeCritic adds two cost terms to the MPPI rollouts:

- Distance-field APF of the classical repulsive form [34], $U = \frac{1}{2}\eta(1/d - 1/d_0)^2$ for $d < d_0$, where d is the Dijkstra distance to the nearest obstacle cell (excluding unknown space), which pushes rollouts off the trapping surface; and
- Free-space gap attraction: repulsion alone does not route the robot around a non-convex (U-shaped) trap, so a ray-cast around the robot finds the opening whose bearing is closest to the goal direction and whose clearance exceeds a threshold, and rollouts whose endpoint heads toward that gap are rewarded – a temporary heading toward an opening (a bearing cost; the goal itself is unchanged), a lightweight form of the free-space subgoal and gap guidance of GP-MPPI [10] and Potential Gap [21].

Because the terms are injected only when entrapped, free-space behavior is unchanged (detect-and-switch, not always-on, as in DRPA-MPPI [12]). In the 2D mirror of Sec. VI-C, which uses the selected bearing as a temporary subgoal rather than through this bearing cost (Appendix), disabling the gap search removes every escape (Sec. VI-C); the gap cost of the C++ critic has not been ablated on its own.

D. CBF safety filter (dynamic obstacles)

To make the barrier relative-degree one in (v, ω) for a unicycle, safety is enforced on a look-ahead point $P = (x, y) + L(\cos \theta, \sin \theta)$, which is fully actuated by (v, ω) through the Jacobian

$$G = \begin{bmatrix} \cos \theta & -L \sin \theta \\ \sin \theta & L \cos \theta \end{bmatrix}. \quad (1)$$

For each obstacle o with $d = P - p_o$ and effective radius $R = r + R_o + m$:

$$h_o = \|d\|^2 - R^2, \quad \dot{h}_o = 2d^\top (Gu - v_o). \quad (2)$$

The continuous-time CBF condition $\dot{h}_o + \alpha h_o \geq 0$ [37], [38], enforced once per control cycle, is linear in u . We solve a small QP (OSQP) [47] that minimizes

$$\|u - u_{\text{nom}}\|^2 + \rho \delta^2 \quad (3)$$

TABLE I

Positioning. (a) ships as a Nav2 controller plugin; (b) online local-minimum escape or avoidance; (c) an explicit safety constraint, filter or backup enforced at run time (for DualGuard, Hamilton–Jacobi reachability; for Soloperto et al., a safe backup trajectory; for Safer Gap, zeroing barrier functions); (d) online adaptation of the CBF class- $\mathcal{K}$ gain. $\checkmark$ = holds; $\circ$ = holds in part: stock Nav2 detects stalls at the stack level and runs generic recoveries rather than a steered escape; the 3Laws Supervisor is integrated with Nav2 as a separate node after the controller, not as a controller plugin; the gap-based planners guide the robot through free-space gaps rather than escaping a detected minimum, and the provable collision-free properties of Potential Gap and Dynamic Gap follow from the planners’ gap construction rather than from an explicit run-time constraint or filter; Kim et al. reduce predicted deadlock through class- $\mathcal{K}$ parameter selection rather than escaping a detected minimum. For the 3Laws Supervisor, (b) is marked absent because neither cited source describes a local-minimum escape mechanism. [†]Concurrent work (posted in 2026). For SE-MPPI, (c) is marked in part because its barrier covers only tracked dynamic obstacles, at a look-ahead point, and its invariance argument is conditional (Sec. IV-E); static structure is handled only by MPPI costs. Its (b) is demonstrated only in the Python 2D mirror, not in a live Nav2 run. No reviewed system holds all four; several hold two. The review behind this table (June 2026, extended in September 2026) covered the Nav2 MPPI critic ecosystem, the Nav2 documentation, the public issue trackers of these projects, and the works cited here; it is not exhaustive.

System	(a)	(b)	(c)	(d)
Nav2 MPPI with progress checker and recoveries [1], [3], [4]	$\checkmark$	$\circ$	–	–
3Laws Supervisor in Nav2 [18], [19]	$\circ$	–	$\checkmark$	–
DRPA-MPPI, GP-MPPI [12], [10]	–	$\checkmark$	–	–
Dynamic Gap, Potential Gap [23], [21]	–	$\circ$	$\circ$	–
Safer Gap [22]	–	$\circ$	$\checkmark$	–
Backup-trajectory MPC [20]	–	$\checkmark$	$\checkmark$	–
MILP-MPC with CBF [†] ; MCBF-QP; predictive MCBF [†] [27], [24], [26]	–	$\checkmark$	$\checkmark$	–
DualGuard, SCBF-MPPI, GS-MPPI, Shield-MPPI [16], [15], [17], [13]	–	–	$\checkmark$	–
CBFKit, DPCBF [35], [36]	–	–	$\checkmark$	–
BR-MPPI [14]	–	–	$\checkmark$	$\checkmark$
Rate-tunable, adaptive, optimal-decay CBFs [30], [29], [39]	–	–	$\checkmark$	$\checkmark$
Deadlock-aware parameter adaptation [28]	–	$\circ$	$\checkmark$	$\checkmark$
SE-MPPI (ours)	$\checkmark$	$\checkmark$	$\circ$	$\checkmark$

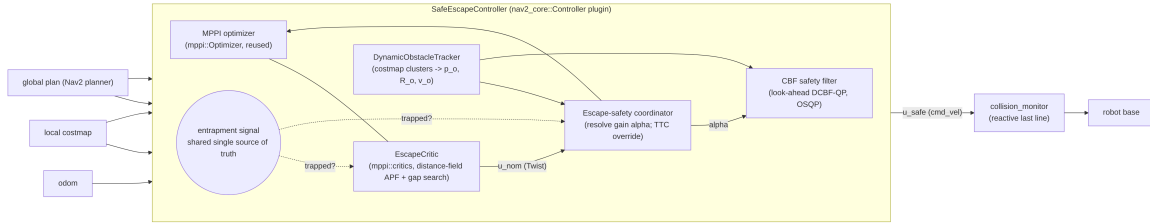


Fig. 1. SE-MPPI architecture / data flow. Global plan, local costmap, and odometry feed the SafeEscapeController plugin: the reused MPPI optimizer (with the EscapeCritic) emits the nominal command, a shared entrapment signal drives both the critic and the escape-safety coordinator, the coordinator resolves the CBF gain α (with TTC override), and the CBF safety filter – fed by the DynamicObstacleTracker – projects the command to `cmd_vel`, which Nav2’s `collision_monitor` guards as a final reactive layer.

subject to the per-obstacle CBF rows (relaxed by a single slack $\delta \geq 0$ for feasibility) and the input box limits. Obstacles are pruned to the nearest few by clearance (surface gap), not center distance. If the QP can only stay safe by relaxing the barrier ($\delta > \epsilon$) or fails, the controller brakes the forward velocity to zero while keeping the QP’s turn (or, on solver failure, the box-clamped nominal turn) – stop rather than drive into an imminent collision. Static structure is intentionally excluded from the CBF (handled by the MPPI obstacle critic and costmap inflation); only obstacles that are moving and small enough to be a movable body are admitted, which we found essential in a real costmap (Sec. VI-B).

Because this condition is checked once per control cycle (a sampled-data CBF), strict between-sample forward invariance would require a sampled-data margin tightening R by an $O(\dot{h}_{\max} \Delta t)$ term; in practice the obstacle-radius margin m absorbs this gap, and we report the empirical safety rate rather than claim continuous-time invariance between samples.

E. Escape-safety coordination: gain scheduling under a conditional invariance lemma

The coordinator resolves the gain:

$$\alpha = \begin{cases} \alpha_{\text{base}} & \text{not entrapped, or } \text{TTC} < \tau \text{ (imminent)} \\ \alpha_{\text{escape}} & \text{entrapped and } \text{TTC} \geq \tau \end{cases} \quad (4)$$

with $\alpha_{\text{escape}} > \alpha_{\text{base}} > 0$, where TTC is the minimum time-to-collision over tracked obstacles under the current command.² Raising α relaxes the constraint $\dot{h} \geq -\alpha h$ – permitting the robot to approach an obstacle faster, as an escape maneuver requires – while the TTC override snaps the gain back to α_{base} (safety first) when a dynamic obstacle is imminent. The next result states that, under

²The released controller’s coordinator additionally holds α at α_{base} whenever the online conformal bound q on obstacle-prediction error exceeds a trust threshold (0.25 m by default; the calibrator is enabled by default), and that bound inflates the effective radius R of Sec. IV-D – the SE-Predict margin noted in Sec. II, not claimed by this paper. An implementation following Eq. (4) alone therefore reproduces the benchmark configurations of Sec. VI-C, not the released defaults.

explicit conditions, this schedule does not by itself give up forward invariance. It is a known property of CBFs with time-varying class- $\mathcal{K}$ terms and follows the same argument as the relative-degree-one case of the invariance result for adaptive CBFs with time-varying penalty functions [29, Thm. 3], which is stated there for Lipschitz-continuous controllers, whereas our switched gain yields a discontinuous command; related time-varying and optimized decay rates appear in [39] and [30]. We restate it with the standard integrating-factor proof for completeness and claim no new mathematics.

Proposition (invariance under a time-varying gain; after [29]). Fix an obstacle j , suppose the safety-filter QP is feasible without slack throughout the interval considered (the shared $\delta(t) = 0$), and assume the continuous-time inequality $\dot{h}_j(t) \geq -\alpha(t) h_j(t)$ holds for almost every $t \geq 0$ – an assumption between samples, since discrete-instant QP feasibility does not by itself imply the intersample inequality (the sampled-data caveat of Sec. IV-D). Let $\alpha : [0, \infty) \rightarrow [\alpha_{\min}, \alpha_{\max}] \subset (0, \infty)$ be any measurable, positive, bounded gain schedule – in particular the coordinator’s piecewise-constant switching between α_{base} and α_{escape} (Eq. 4) and its TTC override. If $h_j(0) \geq 0$, then $h_j(t) \geq 0$ for all $t \geq 0$ – the look-ahead point P does not enter obstacle j ’s inflated disc, as modeled (body safety approximate; see Sec. VII).

Proof. Define the integrating factor $\varphi(t) = \exp(\int_0^t \alpha(s) ds)$, finite and strictly positive because $\alpha \leq \alpha_{\max}$. Then, almost everywhere,

$$\frac{d}{dt} [\varphi(t) h_j(t)] = \varphi(t) [\dot{h}_j(t) + \alpha(t) h_j(t)] \geq 0, \quad (5)$$

so $\varphi(t) h_j(t)$ is non-decreasing, hence $\varphi(t) h_j(t) \geq \varphi(0) h_j(0) = h_j(0) \geq 0$. Since $\varphi(t) > 0$, $h_j(t) \geq 0$ for all t . ■

Positivity and boundedness are stronger than the argument needs: a nonnegative, locally integrable α already makes φ finite and positive. We keep the stronger hypothesis because it matches the schedule of Eq. (4).

Corollary. Under the slack-free and between-sample conditions of the Proposition, forward invariance of $\mathcal{C}_j = \{h_j \geq 0\}$ holds for every positive, bounded gain schedule – in particular for the coordinator’s switching between α_{base} and α_{escape} , and for the TTC override that drops α back to α_{base} . If these conditions hold for every row, $\mathcal{C} = \bigcap_j \mathcal{C}_j$ is invariant; the argument is per-obstacle and gain-agnostic, so sharing a single $\alpha(t)$ across rows is immaterial to the invariance statement itself.

Under these conditions, raising α therefore does not permit $h_j < 0$; it only weakens the lower bound on $\dot{h}_j$, letting the robot approach $\partial \mathcal{C}_j$ faster (bolder detours) inside the same set. This is the design rationale for coordinating rather than stacking the two layers: in this idealized setting, the gain trades caution for maneuverability without giving up invariance. The guarantee is narrow. It concerns the look-ahead point, not the robot body; it covers only the tracked dynamic obstacles under the constant-velocity model, not static structure; it applies to

an obstacle only while that obstacle stays in the tracked, unpruned set $\mathcal{O}$ (an obstacle that slows below the speed threshold for dynamic obstacles, merges into a cluster too large to be a movable body, or is pruned beyond the nearest few by clearance leaves the QP); it assumes the continuous-time inequality between samples; and it requires $\delta = 0$, so it does not cover any cycle in which the QP uses slack. Because the slack is shared, one imminent obstacle can momentarily loosen the margin on every other obstacle in the same QP solve, so we report slack usage empirically: in the 2D benchmark of Sec. VI-C the QP never engages slack on the mover-free families, and uses nonzero slack in 26–34 of 50 narrow-dynamic trials (max 0.47) and 47–48 of 50 dynamic-family trials (max 0.015) per CBF-carrying config (Sec. VI-C1). In cycles with $\delta > \epsilon$ the brake of Sec. IV-D replaces the QP command, and bounded-prediction-error robustness is the subject of follow-up work (Sec. VII).

F. Dynamic-obstacle tracking

The tracker clusters costmap cells above a lethal threshold, associates each current cluster to at most one previous cluster within a gate (so a split/merge cannot manufacture a phantom velocity), and estimates a clamped constant velocity. Its output $\{p_o, R_o, v_o\}$ feeds the coordinator (TTC) and the CBF filter. The constant-velocity model is a deliberate, replaceable baseline (Sec. VII).

V. Implementation (Nav2-Native)

SE-MPPI is built on ROS 2 Jazzy + Nav2 [1]. The package `nav2_se_controller` ships three shared libraries: a plugin-free `se_mppi_core` (entrapment, CBF, coordinator, tracker, repulsion, gap search), the `escape_critc` MPPI critic, and the `safe_escape_controller`. Separating the plugin-free core from the two plugin libraries resolves a `class_loader` factory conflict that otherwise prevents the critic and controller from co-loading.

Two integration details proved essential and are easy to get wrong:

- Critic namespace. Nav2’s CriticManager resolves a critics-list entry NAME to the plugin class `mppi::critics::NAME`. A critic in any other namespace can be unit-tested via `pluginlib` directly but will never load from the critics list. `EscapeCritic` therefore lives in `mppi::critics`.
- Jazzy critic data uses `xtensor`, not the Eigen of the main branch; the critic was written against the installed headers.

The QP uses `osqp-eigen` [47]. The build is reproducible via RoboStack (conda-forge) so the build does not depend on the host OS version (validated on Ubuntu 20.04 with an NVIDIA GPU). The package’s Paper-1 modules (entrapment detector, CBF filter, coordinator, tracker, gap search, repulsion, path progress, and plugin loading) carry 44 unit tests (gtest), plus linters; the wider package additionally carries tests for the follow-up papers’ prediction stack (static/dynamic occupancy classification, trajectory prediction, conformal calibration) and multi-robot

coordination module (SE-Predict, Multi-SE-MPPI), plus the live-injection ABI regression of Sec. VI-B, bringing the algorithm test suite (`test/test_*.cpp`) to 82 TEST cases across 13 files. The code described and evaluated in this paper is the release tagged `paper-v1`; later commits add deployment tests and fixes that are not part of the evaluated code.

VI. Experiments

A. Mechanism validation (standalone 2D)

To validate each mechanism independently of simulator infrastructure, we reimplement the controller’s core math (APF, gap raycast, look-ahead CBF-QP via OSQP [47], α modulation + TTC override, monotone-progress entrapment) as a 2D unicycle and run three scenarios. The 2D code reuses the controller’s formulas but differs from the Nav2 controller in documented ways (listed in the Appendix); in these three scenes its CBF sees all obstacles, including the static wall discs. Measured results:

Three findings: (1) stock MPPI stalls in front of a U-trap while SE-MPPI detects and rounds it; (2) against a crossing dynamic obstacle, the no-CBF baseline collides while the speed-aware CBF filter avoids it; and (3) the coordination plot shows the gain rise $\alpha : 2 \rightarrow 6$ during the escape phase with slack ≈ 0 throughout – i.e., the escape maneuver is admitted without relaxing the barrier, close to the slack-free setting that the Proposition of Sec. IV-E assumes. In these benign scenarios the coordinated and independent variants are both safe; whether coordination yields a quantitative outcome benefit is tested directly in the randomized benchmark of Sec. VI-C, which finds it null in the regimes reachable there (Sec. VI-C1).

These three findings are shown in Fig. 2 (sources in `experiments/prototype/figures/`; generated by `experiments/prototype/run_validation.py`).

B. Live Nav2 + Gazebo integration

We deployed SE-MPPI in a complete live stack (ROS 2 Jazzy, Nav2 – AMCL, costmaps, NavFn, BT navigator, recovery, collision monitor – and Gazebo `tb3_sandbox`) [1] on the target workstation. Confirmed at the level of committed launch logs and trial telemetry: the controller and `EscapeCritic` load and activate from the critics list; the controller produces valid velocity commands identical in form to stock MPPI; and the full lifecycle reaches the active state, sustaining multi-minute drives on a U-trap testbed without a crash or collision after the ABI fix below (the goal was not reached: the committed telemetry records STUCK at 254.5 s and TIMEOUT at 299.8 s, both collision-free, under the host constraints noted below). The escape-injection path has also run live: its first live activation is what exposed the ABI fault below – a fault unreachable from unit tests – and the tensor shapes captured from that event are committed as a regression test, while the controller and critic now log every detection and injection transition (ENTER/EXIT, BEGIN/END) for auditability. No end-to-end Nav2 escape has therefore

been demonstrated: a sustained live entrapment-escape demonstration on this host remains confounded by the host-level artifacts noted below and is deferred to the full-scale benchmark. We validate the live integration to this depth; the controlled quantitative comparison is the 2D benchmark of Sec. VI-C, because a full live A/B on this WSL2 host measures the host (RTF ≈ 0.3 – 0.6 , with TF smear inflating the costmap with phantom cells) rather than the algorithm.

Three integration faults surfaced that only a live stack reveals, each reported as a deployment lesson:

- Static walls in the CBF. Feeding every lethal cluster (including static walls) into the CBF makes the barrier infeasible everywhere and freezes the robot – fixed by scoping the CBF to genuinely dynamic obstacles (Sec. IV-D).
- Compute budget. The per-cycle budget must be matched to the host (horizon preserved, rate and batch reduced) or the loop misses its rate and the optimizer diverges.
- `xtensor` ABI mismatch. The installed `libmppi_controller.so` is compiled with `XTENSOR_USE_XSIMD + AVX2` (an `xsimd::aligned_allocator` element type), whereas our plugin compiled `xtensor` with the default `std::allocator`. Because `CriticData`’s tensors cross the pluginlib boundary by reference, the first `EscapeCritic` write to `data.costs` was an ODR violation that segfaulted the entire Nav2 process on the first live escape injection (3/3 reproducible). The in-process unit tests never caught it – they compile every translation unit with one consistent flag set – and earlier live gates never triggered entrapment, so the injection path had never actually run live. Fixed by compiling the critic against the installed headers’ flags (per-target, since a global AVX2 flip breaks the separately built OSQP–Eigen boundary); a regression test now replays the injection path with live tensor shapes. This is precisely the class of fault a Nav2-native deployment exposes and that a standalone node or a pure-2D study cannot.

C. Randomized benchmark on a Python 2D mirror

Every quantitative result in this section comes from a Python 2D mirror, not from the Nav2 controller. The mirror is a randomized 2D benchmark that re-implements the controller’s primitives in NumPy and OSQP [47] – the MPPI sampler, the APF repulsion, the gap raycast, the look-ahead-point CBF-QP, the α coordination with TTC override, and the monotone-progress entrapment detector – and drives them across randomized scenarios so that success, collision, and clearance carry confidence intervals and paired significance tests. As in the released controller and the deployment lesson of Sec. VI-B, the CBF and TTC see only the dynamic obstacles, not static walls. Like the controller, the mirror enforces the barrier on the look-ahead point rather than the full body; unlike

TABLE II
Standalone 2D mechanism validation (measured).

Scenario / config	reached	collided	time	min-clear (m)
U-trap / Stock MPPI	× (stuck)	—	—	0.33
U-trap / SE-MPPI (escape)	✓	—	27.7 s	0.32
Dynamic / No CBF	×	collision	—	−0.00
Dynamic / SE-MPPI (CBF)	✓	safe	18.6 s	0.01
Coordination: SE-MPPI vs. independent [†]	✓	safe	27.7 s	0.32

[†]SE-MPPI (F) and the independent stack (E) return identical trajectories and metrics in this benign scenario; the quantitative null test is Table IV.

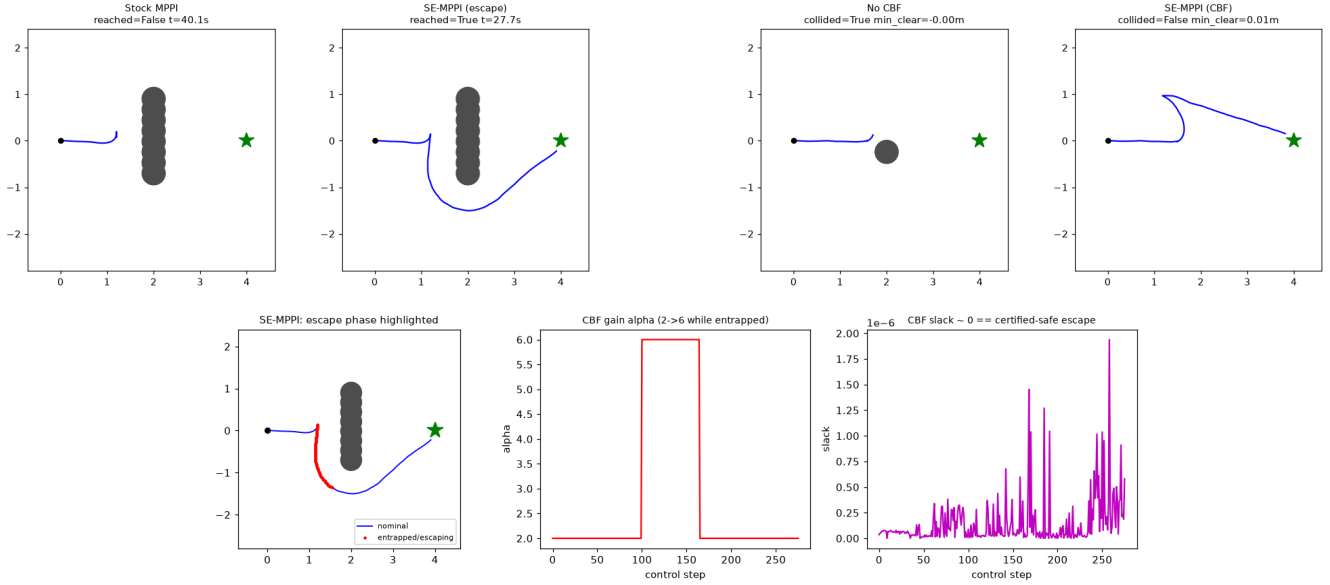


Fig. 2. Standalone 2D mechanism validation (trajectories/traces only; no benchmark metrics). (a) U-trap escape: stock MPPI (left) stalls at the trap mouth ($x \approx 1.2$, no progress) while SE-MPPI (right) detects the stalled progress and rounds the U-shaped obstacle to the goal via the gap-attraction subgoal. (b) Dynamic-obstacle avoidance: against a crossing obstacle the no-CBF baseline (left) collides while the speed-aware look-ahead CBF filter (right) projects the command to a collision-free path. (c) Escape-safety coordination: during the escape phase the coordinated gain rises $\alpha : 2 \rightarrow 6$ while the QP slack stays ≈ 0 throughout – the escape maneuver is admitted without relaxing the barrier, close to the slack-free setting that the Proposition of Sec. IV-E assumes.

it, the mirror uses distance-to-goal progress and a free-space gap subgoal in place of a global path, and computes geometry analytically against circular obstacles instead of on a costmap; the Appendix lists every difference. Two differences matter most for reading the results: the mirror uses the selected gap as a temporary subgoal that replaces the goal, whereas the C++ critic keeps the goal and the path-following critics and adds a bearing cost toward the selected gap (Sec. IV-C), and the mirror gives the CBF and the TTC the true obstacle states, so the costmap tracker of Sec. IV-F is not exercised. We use this 2D surface rather than a live Gazebo full matrix because at RTF 0.3–0.6 under CPU contention the live host measures itself, not the algorithm (Sec. VI-B); the full-scale physics benchmark is deferred to future work (Sec. VII).

Setup. Four seed-deterministic scenario families exercise the two weaknesses and their conjunction: utrap (a finite U/C pocket with the goal behind the back wall – a static local minimum), clutter (a random field of 5–9 circular obstacles with a guaranteed footprint-clear path), dynamic (1–2 pedestrian-like crossing movers timed

to reach the robot’s path), and narrowdyn (the utrap pocket plus movers sweeping the pocket mouth during the escape window – the narrow-and-dynamic coincidence the coordination targets). Six ablation configs toggle the mechanisms: A stock (no escape/CBF), C escape only, D CBF only, E escape+CBF with $\alpha_{\text{escape}} = \alpha_{\text{base}}$ (independent), F full SE-MPPI ($\alpha : 2 \rightarrow 6$ + TTC override, coordinated), and F[−] (F with the gap search off). Config A is stock MPPI without Nav2’s progress checker and behavior-tree recoveries [3], [4], so it is a controller-level baseline, not the full Nav2 stack. Each family is a pure function of an integer seed, so every config is evaluated on an identical scenario set, as the paired McNemar design requires: 4 families $\times$ 50 seeds $\times$ 6 configs = 1,200 paired trials, 40 s sim budget each; these are 200 seeded worlds, each run under all six configs, so the trials are paired rather than independent. Success carries a Wilson 95% CI [48]; the primary test is McNemar’s [49] on paired outcomes; continuous metrics use the Mann-Whitney U test [50] with Cliff’s δ [51], an unpaired comparison that ignores the pairing, with time-to-goal and path length taken over

TABLE III

Randomized benchmark on the Python 2D mirror (not the Nav2 controller): success rate in % (Wilson 95% CI), $N = 50$ per cell. On the trap families (utrap, narrowdyn) the escape configs C, E, F separate from stock A, CBF-only D, and no-gap F^- ; F^- collapses to 0%; E and F coincide except on narrow-dyn, where they differ by one trial (64% vs 62%).

Config	U-trap	Clutter	Dynamic	Narrow-dyn
A stock	0 [0,7]	52 [39,65]	80 [67,89]	0 [0,7]
C escape	90 [79,96]	62 [48,74]	76 [63,86]	78 [65,87]
D CBF	0 [0,7]	52 [39,65]	82 [69,90]	0 [0,7]
E indep.	88 [76,94]	62 [48,74]	78 [65,87]	64 [50,76]
F SE-MPPI	88 [76,94]	62 [48,74]	78 [65,87]	62 [48,74]
F^- no-gap	0 [0,7]	52 [39,65]	84 [71,92]	0 [0,7]

TABLE IV

Key paired contrasts (McNemar, Holm-adjusted; b/c discordant pairs, b = baseline wins, c = F wins), Python 2D mirror. The escape contrast is decisive on the trap families; the coordination contrast ($E \rightarrow F$) is null everywhere.

Family	Escape (stock $\rightarrow$ F)		Coord. ($E \rightarrow F$)	
	b/c	p_{adj}	b/c	p_{adj}
U-trap	0/44	$1.4 \times 10^{-9} *$	0/0	1
Clutter	0/5	1	0/0	1
Dynamic	3/2	1	0/0	1
Narrow-dyn	0/31	$1.1 \times 10^{-6} *$	1/0	1

successful trials only, so we read these as descriptive; all p -values are Holm–Bonferroni-corrected [52] within each family.

The escape layer is the measured contribution in the mirror, and the mirror’s gap search is load-bearing. On both trap families the effect is decisive and survives Holm correction (Table IV): success rises from 0% (stock A, CBF-only D, and no-gap F^- all time out in the pocket) to 88–90% on utrap (44/50 discordant pairs, $p_{adj} = 1.4 \times 10^{-9}$) and to 62–78% on narrowdyn (31/50 discordant, $p_{adj} = 1.1 \times 10^{-6}$). That F^- scores 0% on both trap families (Table III) shows that, in the mirror, the detect-and-switch mechanism escapes only with the free-space gap subgoal: APF and gain modulation without a routed opening do not escape. There the gap acts as a temporary subgoal (Appendix), a form close to the subgoal guidance of GP-MPPI [10]; whether the C++ critic’s bearing cost toward the gap is equally load-bearing has not been tested. On clutter the escape adds 10 pp (52% $\rightarrow$ 62%, not significant after Holm) at the cost of ≈ 2.4 s longer average time-to-goal (over successful trials), and on the pure-dynamic family – which contains no trap – it is appropriately neutral (Table IV, dynamic stock $\rightarrow$ F: $p = 1$). Fig. 3 gives the full per-family picture.

1) Escape–safety coordination is null in this regime: The load-bearing contrast for the C2 contribution is E (independent) vs. F (coordinated): identical mechanisms differing only in whether the CBF gain is coordinated with the escape phase. Across all 200 paired scenarios this contrast is null (Table IV, Fig. 4): the discordant counts are 0/0, 0/0, 0/0, and 1/0, McNemar $p = 1$ in every family, and every continuous effect size is $|\delta| \leq 0.03$. The gain

does engage as designed ($\alpha_{max} = 6$ during escape, TTC override active). The committed per-trial slack telemetry (slack_max in trials.csv; per-config rates in stats.json and tables.md) locates where the barrier actually works. On the mover-free families (utrap, clutter) the dynamic-obstacle CBF sees no obstacles at all: slack is zero in every trial, E and F issue identical commands, and the null there is structural. On narrowdyn the barrier does engage and even relaxes – the QP uses nonzero slack in 34/50 trials for E and F (26/50 for D and F^- ; max slack 0.47) – and on dynamic in 47–48/50 trials (max 0.015), yet the E-vs-F contrast stays null (1/0 discordant, $p = 1$). The correct reading is therefore not that the barrier never binds, but that the outcomes are insensitive to the α schedule: where the barrier is quiescent the two configs coincide exactly, and where it is active the base and escape gains admit escape maneuvers whose outcomes are indistinguishable at $N = 50$. (The mechanism plot of Sec. VI-A, “ $\alpha : 2 \rightarrow 6$ with slack ≈ 0 ,” shows the benign end of this spectrum.) We report this honestly: in this 2D benchmark the coordination produces no measurable outcome difference over independent escape+CBF. The result is consistent with – and delimits – the Proposition of Sec. IV-E: coordination is a mechanism with a conditional invariance property, not, in these scenarios, an outcome improvement. An outcome-level benefit would require a regime where the choice of α changes which escape maneuvers are admitted at outcome-deciding moments – faster or persistent movers, tighter margins, a dynamic gap the escape route must thread while the barrier is live; this generator’s pedestrian-like movers produce barrier engagement but not that sensitivity. Constructing and measuring that regime is future work (Sec. VII); we claim no coordination benefit we have not measured.

Two CBF observations bear on Sec. VII. Against body-grazing crossers (dynamic) the look-ahead-point filter leaves the collision rate essentially unchanged (20% $\rightarrow$ 16–18%; descriptive – the committed analysis carries no paired collision-outcome test). And in the cramped narrowdyn pocket the CBF configs show a higher collision rate than their no-CBF counterparts (18% $\rightarrow$ 32%, descriptive) and cost ≈ 16 pp success (78% $\rightarrow$ 62%; success McNemar F vs. C: $b/c = 8/0$, raw $p = 0.008$, Holm-adj. 0.109 – directionally consistent though not significant after correction at $N = 50$). Both are discussed as limitations in Sec. VII.

Full-scale benchmark (future work). The live full-matrix comparison this section stands in for is specified but deferred: an identical Nav2 stack and differential robot across three tiers – BARN (static difficulty) [53], [54], DynaBARN (dynamic) [55], and HuNavSim (human-aware) [56] – with stock MPPI [1], [2], DWB [32], RPP [33], TEB [31], an always-on variant of our escape critic, and a CBF-only baseline (Shield-style [13]), scored by success, collision, time-to-goal, path length, clearance, and per-cycle compute under the same McNemar / Mann–Whitney + Holm protocol. DRPA-MPPI itself [12], GP-MPPI [10], a gap-based planner [22], and an adaptive-gain

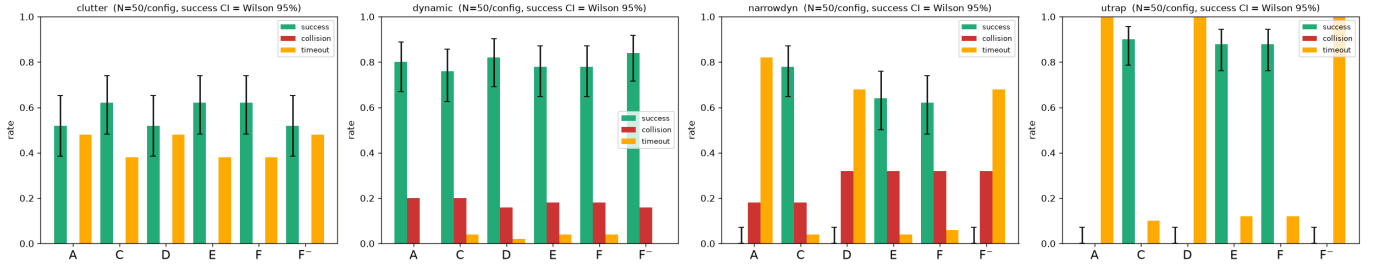


Fig. 3. Randomized benchmark on the Python 2D mirror: success / collision / timeout per family across configs A–F[−] (Wilson 95% error bars on success, $N = 50$). On the trap families (utrap, narrowdyn) the escape configs (C = escape-only, E = independent escape+CBF, F = coordinated) separate cleanly from stock A, CBF-only D, and no-gap F[−], which time out at 0% success. E and F coincide in every family except narrow-dynamic, where they differ by a single discordant trial (McNemar $p = 1$).

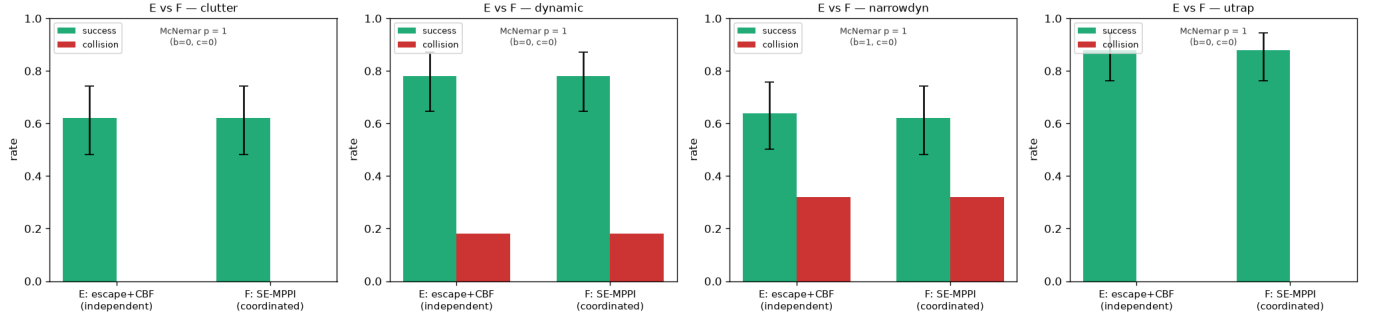


Fig. 4. The coordination contrast E vs. F per family in the Python 2D mirror, annotated with the McNemar p -value. The independent (E) and coordinated (F) stacks are statistically indistinguishable in every family ($p = 1$) – the null result of Sec. VI-C1.

CBF [28] have not been run either; we make no comparison to them. Running the protocol on a real-GPU host is the remaining step (Sec. VII).

VII. Limitations

- Full-scale physics benchmark and baselines are future work. The quantitative comparison in this paper is the randomized benchmark on the Python 2D mirror of Sec. VI-C, which re-implements the controller’s primitives but is not the live controller. The full-scale 3D physics benchmark specified there (BARN/DynaBARN/HuNavSim against the deployable baselines) is not yet run: the live host available to us (WSL2, RTF ≈ 0.3 – 0.6 , with TF smear inflating the costmap with phantom cells) measures itself rather than the controller (Sec. VI-B), so it needs a real-GPU host. The live entrapment-and-escape reproduction is deferred for the same reason; no live run has yet reached its goal, so controller-level escape in Nav2 is not demonstrated. The planned evaluation also adds the baselines that the prior work of Sec. II calls for: stock MPPI with Nav2’s progress checker and behavior-tree recoveries enabled [3], [4], DRPA-MPPI [12], a GP-MPPI-style subgoal escape [10], a gap-based planner [22], and an adaptive-gain CBF [28].
- Mirror versus controller. Every quantitative result comes from the Python 2D mirror, whose progress signal, gap use, geometry, obstacle knowledge, and

costs differ from the C++ controller (Appendix). In particular, the load-bearing gap result concerns the mirror’s temporary subgoal, not the critic’s bearing cost toward the gap, and the costmap tracker is not exercised.

- Statistics. The 1,200 trials are 200 seeded worlds under six configs, not independent scenarios. Success is tested with paired McNemar tests, but the continuous metrics use unpaired Mann–Whitney tests, with time-to-goal and path length conditioned on success, so they are descriptive; a paired analysis that handles failures explicitly belongs to the full-scale benchmark.
- Coordination benefit is unmeasured, not established. The escape–safety coordination (C2) has the conditional invariance property of Sec. IV-E but produced no outcome difference over an independent escape+CBF stack: outcomes were insensitive to the α schedule even where the barrier engages (Sec. VI-C1), and the idea of adapting the CBF gain online is not new [28], [14], [30]. Demonstrating an outcome-level benefit requires a regime where the α choice decides which escapes are admitted (faster/persistent movers, tighter dynamic gaps); constructing and measuring it is future work.
- Braking is not a safe fallback when cornered. In the cramped narrow-dynamic pocket the infeasible-QP fallback ($v = 0$) leaves the robot in the path of a sweeping mover, and the CBF configs measured a higher collision rate than their no-CBF counterparts

(18% $\rightarrow$ 32%; Sec. VI-C). A cornered robot needs an evasive fallback, not a stop.

- Look-ahead-point vs. full-body safety. The Proposition of Sec. IV-E concerns the look-ahead point P (the relative-degree-one output), not the full robot body. Body safety is therefore approximate: the offset $L \approx r$ places P at the robot's leading edge, and the costmap obstacle critic plus collision_monitor cover the residual. A full-body guarantee would require inflating the margin by L , which we deliberately avoid because it over-constrains sub-meter gaps (BARN's maximum clearance is ≈ 0.9 m) and would sharply degrade the success rate. This gap is measured: on the dynamic family the look-ahead filter left body-grazing collisions essentially unchanged (20% $\rightarrow$ 16–18%, descriptive; Sec. VI-C).
- Sampled-data and slack caveats. Invariance between samples is assumed in the Proposition of Sec. IV-E, not proven, and the shared slack couples the rows of one solve; we report slack usage instead of claiming an unconditional guarantee. Static structure is outside the barrier altogether, and in cycles with $\delta > \epsilon$ the braking fallback replaces the QP command.
- Constant-velocity tracker. The dynamic-obstacle model evaluated in this paper is a replaceable constant-velocity (CV) baseline with rotating/accelerating agents mispredicted and prediction uncertainty unquantified, and an obstacle that leaves the tracked, unpruned set $\mathcal{O}$ also leaves the barrier (Sec. IV-E). The released controller has since added (i) occupancy-persistence static/dynamic classification (no wall-freeze on association jitter), (ii) persistent tracks with least-squares CV/CVCA horizon prediction, and (iii) an online conformal bound q that inflates the CBF effective radius (time-varying radius) and gates the escape gain on prediction trust. These additions are the subject of the follow-up paper (SE-Predict); none of them is present in the 2D mirror behind every quantitative result reported here, this paper's claims are isolated from them, and the follow-up paper's own no-conformal ablation will isolate them there.
- TTC is a 1-D approximation; tight spaces can induce over-rotation.
- Real-time. Per-call QP cost grows with the obstacle budget (capped by clearance pruning).
- Localization in symmetric/narrow maps can jump and perturb progress estimation – an environment property, mitigated in structured benchmarks.

VIII. Conclusion

SE-MPPI is a system that packages local-minima escape and control-barrier-function safety, two problems that several prior works also address together outside Nav2, in a single Nav2 controller plugin: a shared entrapment signal drives both a sampling-time escape critic and an output-time CBF filter, and a coordinated gain raises the barrier's class- $\mathcal{K}$ gain during escape, which, by a standard argument

for time-varying gains, keeps the look-ahead point's safe set with respect to the tracked dynamic obstacles forward invariant, assuming the barrier inequality holds between control updates and the QP uses no slack. A randomized, paired 1,200-trial benchmark run on a Python 2D mirror of the controller's primitives shows the online detection-and-escape layer is the decisive contribution – 0% to 88–90% success on the static U-trap family and 0% to 62–78% on the narrow-dynamic family, with the mirror's free-space gap subgoal load-bearing – while the escape-safety coordination is empirically indistinguishable from an independent escape+CBF stack in the regimes we could test (the null of Sec. VI-C1), which we report openly. We fixed three integration faults that only a live stack reveals – static-wall CBF infeasibility, host-matched compute budgeting, and an xtensor ABI mismatch that segfaulted Nav2 on the first live escape injection – and confirmed the controller loads, activates, and runs in a full ROS 2 Jazzy + Nav2 + Gazebo deployment, although no live run has yet reached its goal, with the live injection event that exposed the ABI fault preserved as a committed regression test. The remaining steps are a full-scale 3D physics benchmark on a real-GPU host, with Nav2's own stall recovery, DRPA-MPPI, subgoal and gap-based baselines, and a successful end-to-end Nav2 escape. Beyond this paper, the codebase has grown the uncertainty-calibrated prediction stack (SE-Predict: classification, horizons, conformal bounds – paper 2) and the multi-robot reciprocal coordination layer (Multi-SE-MPPI – paper 3); both reuse this paper's coordination thesis and the shared evaluation harness.

Appendix Reproducibility

All paths below are relative to the code release of contribution C4 (footnote in Sec. I).

Software. ROS 2 Jazzy, Nav2, and Gazebo are installed via RoboStack (conda channels robostack-jazzy + conda-forge) by the committed script scripts/setup_ros2_env.sh, which also pulls osqp-eigen/eigen for the CBF-QP [47] and the cxx-compiler/cmake/ninja toolchain, so the build does not depend on the host OS version (Sec. V).

Benchmark regeneration. Every number in Sec. VI-C is generated from the committed per-trial CSV by committed scripts, never transcribed by hand. The commands (from docs/papers/2026_2d-benchmark-results.md, re-wrapped to column width; the committed data uses $N=50$):

```
export MAMBA_ROOT_PREFIX=$HOME/micromamba
micromamba run -n ros2 python3 \
  -m experiments.benchmark2d.runner \
  --families utrap clutter dynamic narrowdyn \
  -n <N> --workers 6 --max-steps 400
micromamba run -n ros2 python3 \
  -m experiments.benchmark2d.report # tables + stats
micromamba run -n ros2 python3 -c \
  "from experiments.benchmark2d import figures, \
  aggregate; \
  r=aggregate.aggregate( \
    'experiments/results_2d/trials.csv', \
```

```
'experiments/results_2d'); \
figures.plot_all(r['summary'], r['stats'], \
'experiments/results_2d/figures')
```

Scenarios and seeds. The benchmark is 4 families $\times$ 50 seeds $\times$ 6 configs = 1,200 paired trials (seeds 0–49 per family, 400 steps $\times$ 0.1 s budget each). Each scenario is a pure function of its integer seed: the generators in `experiments/benchmark2d/scenarios.py` seed numpy generators with a deterministic per-family offset plus the trial seed (10^6 utrap, 2×10^6 clutter, 3×10^6 dynamic, 4×10^6 narrowdyn), and the MPPI noise stream uses the same trial seed (`experiments/benchmark2d/runner.py`), so every config sees byte-identical worlds and rollout noise – the common-random-numbers setup the paired McNemar design requires. The six config toggle sets are `experiments/benchmark2d/configs.py`; the ablation overlays for the deferred live full-scale protocol are `experiments/configs/ablations.yaml`.

2D mirror versus the Nav2 controller. The 2D mirror (`experiments/prototype/se_mppi_proto.py`, driven by `experiments/benchmark2d/rollout.py`) reuses the controller’s formulas but differs in the following documented ways.

- Progress signal. Without a global path, entrapment is declared when the best distance-to-goal has not improved by 10^{-3} m for $W = 15$ cycles, instead of the stall of the furthest-reached path index $k_{\max}$.
- Gap use. The selected gap bearing places a temporary subgoal 2.5 m along the opening, replacing the goal in the MPPI cost while entrapped, in lieu of Nav2’s global routing and the critic’s bearing cost toward the gap; the gap choice carries a hysteresis term toward the previous bearing so that symmetric openings do not flip every cycle.
- Geometry. Obstacles, including walls built from chains of discs, are circles; APF distances and ray lengths are computed analytically against them rather than on a costmap distance field.
- Obstacle knowledge. The CBF and TTC receive the true positions and velocities of the moving obstacles, so the tracker of Sec. IV-F is not exercised.
- Costs. The MPPI cost is a goal-distance, path-progress, and obstacle-proximity cost rather than Nav2’s critic stack, and the parameters differ from the released controller’s.
- Mechanism scenes. Table II comes from `run_validation.py`, whose CBF sees all obstacles, including the static wall discs; the benchmark mirror restricts the CBF to movers, as the Nav2 controller does.

Data and parameters. Raw per-trial data: `experiments/results_2d/trials.csv`; aggregates and statistics: `summary.csv`, `stats.json`, `tables.md` (same directory, all auto-generated). The full released-controller parameter set is `src/nav2_se_controller/config/nav2_se_controller_params.yaml`.

References

- [1] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, “The Marathon 2: A Navigation System,” in 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2020. [Online]. Available: <https://arxiv.org/abs/2003.00368>
- [2] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, “Information Theoretic Model Predictive Control: Theory and Applications to Autonomous Driving,” 2017. [Online]. Available: <https://arxiv.org/abs/1707.02342>
- [3] Nav2 contributors, “nav2_controller, branch jazzy: controller_server.cpp and simple_progress_checker.cpp,” Nav2 source code, GitHub repository `ros-navigation/navigation2`, 2026, accessed 2026-09-30. [Online]. Available: https://github.com/ros-navigation/navigation2/tree/jazzy/nav2_controller
- [4] —, “Detailed Behavior Tree Walkthrough,” Nav2 documentation, 2026, accessed 2026-09-30. [Online]. Available: https://docs.nav2.org/rolling/getting_started/nav2_behavior_trees/detailed_behavior_tree_walkthrough/detailed_behavior_tree_walkthrough/
- [5] I. S. Mohamed, K. Yin, and L. Liu, “Autonomous Navigation of AGVs in Unknown Cluttered Environments: log-MPPI Control Strategy,” IEEE Robotics and Automation Letters, 2022. [Online]. Available: <https://arxiv.org/abs/2203.16599>
- [6] Z. Wang, O. So, J. Gibson, B. Vlahov, M. S. Gandhi, G.-H. Liu, and E. A. Theodorou, “Variational Inference MPC using Tsallis Divergence,” 2021. [Online]. Available: <https://arxiv.org/abs/2104.00241>
- [7] K. Honda, N. Akai, K. Suzuki, M. Aoki, H. Hosogaya, H. Okuda, and T. Suzuki, “Stein Variational Guided Model Predictive Path Integral Control: Proposal and Experiments with Fast Maneuvering Vehicles,” 2023. [Online]. Available: <https://arxiv.org/abs/2309.11040>
- [8] E. Trevisan and J. Alonso-Mora, “Biased-MPPI: Informing Sampling-Based Model Predictive Control by Fusing Ancillary Controllers,” IEEE Robotics and Automation Letters, 2024, arXiv:2401.09241. [Online]. Available: <https://arxiv.org/abs/2401.09241>
- [9] J. Sacks and B. Boots, “Learning Sampling Distributions for Model Predictive Control,” in Proceedings of the 6th Conference on Robot Learning (CoRL), 2022. [Online]. Available: <https://arxiv.org/abs/2212.02587>
- [10] I. S. Mohamed, M. Ali, and L. Liu, “GP-Guided MPPI for Efficient Navigation in Complex Unknown Cluttered Environments,” in 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2023, pp. 7463–7470. [Online]. Available: <https://arxiv.org/abs/2307.04019>
- [11] T. Fuke, M. Endo, K. Honda, and G. Ishigami, “Towards Local Minima-free Robotic Navigation: Model Predictive Path Integral Control via Repulsive Potential Augmentation,” in 2025 IEEE/SICE International Symposium on System Integration (SII), 2025, pp. 1112–1117. [Online]. Available: <https://arxiv.org/abs/2410.11379>
- [12] —, “DRPA-MPPI: Dynamic Repulsive Potential Augmented MPPI for Reactive Navigation in Unstructured Environments,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.20134>
- [13] J. Yin, C. Dawson, C. Fan, and P. Tsiotras, “Shield Model Predictive Path Integral: A Computationally Efficient Robust MPC Approach Using Control Barrier Functions,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.11719>
- [14] H. Parwana, T. Kim, K. Long, B. Hoxha, H. Okamoto, G. Fainekos, and D. Panagou, “BR-MPPI: Barrier-Rate Guided MPPI for Enforcing Multiple Inequality Constraints with Learned Signed Distance Fields,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.07325>
- [15] H. Parwana, M. Black, G. Fainekos, B. Hoxha, H. Okamoto, and D. Prokhorov, “Model Predictive Path Integral Methods with Reach-Avoid Tasks and Control Barrier Functions,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.13693>
- [16] J. Borquez, L. Raus, Y. U. Ciftci, and S. Bansal, “DualGuard MPPI: Safe and Performant Optimal Control by Combining Sampling-Based MPC and Hamilton-Jacobi Reachability,” IEEE Robotics and Automation Letters, vol. 10, no. 7, 2025. [Online]. Available: <https://arxiv.org/abs/2502.01924>

- [17] P. Rabiee and J. B. Hoagg, "Guaranteed-Safe MPPI Through Composite Control Barrier Functions for Efficient Sampling in Multi-Constrained Robotic Systems," 2024. [Online]. Available: <https://arxiv.org/abs/2410.02154>
- [18] Nav2 contributors, "Enhanced Safety for Nav2 using 3Laws Supervisor," Nav2 documentation, 2026, accessed 2026-09-30. [Online]. Available: https://docs.nav2.org/rolling/tutorials/general_tutorials/using_3laws_supervisor/using_3laws_supervisor/
- [19] ACCESS Newswire, "Open Navigation and 3Laws Demonstrate Capabilities of Nav2 and 3Laws Supervisor Integration," Press release, 18 December 2025, as syndicated by The Globe and Mail, 2025, accessed 2026-09-30. [Online]. Available: <https://www.theglobeandmail.com/investing/markets/markets-news/ACCESS%20Newswire/36691510/open-navigation-and-3laws-demonstrate-capabilities-of-nav2-and-3laws-supervisor-integration/>
- [20] R. Soloperto, A. Mesbah, and F. Allgöwer, "Safe Exploration and Escape Local Minima With Model Predictive Control Under Partially Unknown Constraints," *IEEE Transactions on Automatic Control*, vol. 68, no. 12, pp. 7530–7545, 2023. [Online]. Available: <https://arxiv.org/abs/2205.03614>
- [21] R. Xu, S. Feng, and P. A. Vela, "Potential Gap: Using Reactive Policies to Guarantee Safe Navigation," *arXiv preprint arXiv:2103.11491*, 2021. [Online]. Available: <https://arxiv.org/abs/2103.11491>
- [22] S. Feng, A. Abuaish, and P. A. Vela, "Safer Gap: A Gap-based Local Planner for Safe Navigation with Nonholonomic Mobile Robots," *arXiv preprint arXiv:2303.08243*, 2023. [Online]. Available: <https://arxiv.org/abs/2303.08243>
- [23] M. Asselmeier, D. Ahuja, A. Zaro, A. Abuaish, Y. Zhao, and P. A. Vela, "Dynamic Gap: Safe Gap-based Navigation in Dynamic Environments," in 2025 IEEE International Conference on Robotics and Automation (ICRA), 2025, pp. 12870–12876. [Online]. Available: <https://arxiv.org/abs/2210.05022>
- [24] Y. Xue and N. Figueroa, "No Minima, No Collisions: Combining Modulation and Control Barrier Function Strategies for Feasible Dynamic Collision Avoidance," *arXiv preprint arXiv:2502.14238*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.14238>
- [25] M. F. Reis, A. P. Aguiar, and P. Tabuada, "Control Barrier Function-Based Quadratic Programs Introduce Undesirable Asymptotically Stable Equilibria," *IEEE Control Systems Letters*, vol. 5, no. 2, pp. 731–736, 2021. [Online]. Available: <https://doi.org/10.1109/LCSYS.2020.3004797>
- [26] Y. Xue, Z. Zhang, K. Åkesson, and N. Figueroa, "Proactive Local-Minima-Free Robot Navigation: Blending Motion Prediction With Safe Control," *IEEE Robotics and Automation Letters*, vol. 11, no. 4, pp. 5096–5103, 2026. [Online]. Available: <https://arxiv.org/abs/2601.10233>
- [27] Y.-H. Chen, S. Ghorl, A. Adil, E. Feron, and C. Belta, "Long-Horizon Geometry-Aware Navigation among Polytopes via MILP-MPC and Minkowski-Based CBFs," *arXiv preprint arXiv:2604.00162*, 2026. [Online]. Available: <https://arxiv.org/abs/2604.00162>
- [28] T. Kim, R. I. Kee, and D. Panagou, "Learning to Refine Input Constrained Control Barrier Functions via Uncertainty-Aware Online Parameter Adaptation," in 2025 IEEE International Conference on Robotics and Automation (ICRA), 2025, pp. 3868–3875. [Online]. Available: <https://arxiv.org/abs/2409.14616>
- [29] W. Xiao, C. Belta, and C. G. Cassandras, "Adaptive Control Barrier Functions for Safety-Critical Systems," *arXiv preprint arXiv:2002.04577*, 2020. [Online]. Available: <https://arxiv.org/abs/2002.04577>
- [30] H. Parwana and D. Panagou, "Rate-Tunable Control Barrier Functions: Methods and Algorithms for Online Adaptation," in 2025 American Control Conference (ACC), 2025, pp. 275–282. [Online]. Available: <https://arxiv.org/abs/2303.12966>
- [31] C. Rösmann, W. Feiten, T. Wösch, F. Hoffmann, and T. Bertram, "Trajectory Modification Considering Dynamic Constraints of Autonomous Robots," in *Proceedings of ROBOTIK 2012; 7th German Conference on Robotics*. VDE Verlag, 2012, pp. 74–79. [Online]. Available: <https://ieeexplore.ieee.org/document/6309484>
- [32] D. Fox, W. Burgard, and S. Thrun, "The Dynamic Window Approach to Collision Avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.
- [33] S. Macenski, S. Singh, F. Martin, and J. Gines, "Regulated Pure Pursuit for Robot Path Tracking," *Autonomous Robots*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.20026>
- [34] O. Khatib, "Real-Time Obstacle Avoidance for Manipulators and Mobile Robots," *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986. [Online]. Available: <https://doi.org/10.1177/027836498600500106>
- [35] M. Black, G. Fainekos, B. Hoxha, H. Okamoto, and D. Prokhorov, "CBFKit: A Control Barrier Function Toolbox for Robotics Applications," 2024. [Online]. Available: <https://arxiv.org/abs/2404.07158>
- [36] H. K. Park, T. Kim, and D. Panagou, "Beyond Collision Cones: Dynamic Obstacle Avoidance for Nonholonomic Robots via Dynamic Parabolic Control Barrier Functions," 2025, accepted to IEEE International Conference on Robotics and Automation (ICRA) 2026. [Online]. Available: <https://arxiv.org/abs/2510.01402>
- [37] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control Barrier Function Based Quadratic Programs for Safety Critical Systems," *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017. [Online]. Available: <https://doi.org/10.1109/TAC.2016.2638961>
- [38] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control Barrier Functions: Theory and Applications," in 2019 18th European Control Conference (ECC), 2019, pp. 3420–3431. [Online]. Available: <https://arxiv.org/abs/1903.11199>
- [39] J. Zeng, B. Zhang, Z. Li, and K. Sreenath, "Safety-Critical Control using Optimal-decay Control Barrier Function with Guaranteed Point-wise Feasibility," in 2021 American Control Conference (ACC), 2021, pp. 3856–3863. [Online]. Available: <https://arxiv.org/abs/2103.12375>
- [40] R. Walia and K. Leahy, "Conflict-Aware Switching for CBF-CLF-Based Multi-Goal Navigation," *arXiv preprint arXiv:2606.21577*, 2026, cDC 2026 submission copy. [Online]. Available: <https://arxiv.org/abs/2606.21577>
- [41] H. Paudel, B. Joshi, D. R. Madaai, A. Bartaula, B. Rimal, and S. Neupane, "A Closed-Form Dual-Barrier CBF Safety Filter for Holonomic Robots on Incrementally Built Occupancy Grid Maps," *arXiv preprint arXiv:2605.05182*, 2026. [Online]. Available: <https://arxiv.org/abs/2605.05182>
- [42] H. Zhou, Y. Zhang, and W. Luo, "Safety-Critical Control with Uncertainty Quantification using Adaptive Conformal Prediction," in 2024 American Control Conference (ACC), 2024. [Online]. Available: <https://arxiv.org/abs/2407.03569>
- [43] J. Gonzales, K. Mizuta, K. Leung, and L. J. Ratliff, "Safe Probabilistic Planning for Human-Robot Interaction using Conformal Risk Control," 2026. [Online]. Available: <https://arxiv.org/abs/2603.10392>
- [44] L. Lindemann, M. Cleaveland, G. Shim, and G. J. Pappas, "Safe Planning in Dynamic Environments using Conformal Prediction," 2022. [Online]. Available: <https://arxiv.org/abs/2210.10254>
- [45] A. Dixit, L. Lindemann, S. Wei, M. Cleaveland, G. J. Pappas, and J. W. Burdick, "Adaptive Conformal Prediction for Motion Planning among Dynamic Agents," 2022. [Online]. Available: <https://arxiv.org/abs/2212.00278>
- [46] L. Busellato, F. Cunico, D. Dall'Alba, M. Emporio, A. Giachetti, R. Muradore, and M. Cristani, "Uncertainty Aware-Predictive Control Barrier Functions: Safer Human Robot Interaction through Probabilistic Motion Forecasting," 2025. [Online]. Available: <https://arxiv.org/abs/2508.20812>
- [47] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "OSQP: An Operator Splitting Solver for Quadratic Programs," *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, 2020. [Online]. Available: <https://arxiv.org/abs/1711.08013>
- [48] E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927. [Online]. Available: <https://doi.org/10.1080/01621459.1927.10502953>
- [49] Q. McNemar, "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages,"

- Psychometrika, vol. 12, no. 2, pp. 153–157, 1947. [Online]. Available: <https://doi.org/10.1007/BF02295996>
- [50] H. B. Mann and D. R. Whitney, “On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other,” *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947. [Online]. Available: <https://doi.org/10.1214/aoms/1177730491>
 - [51] N. Cliff, “Dominance statistics: Ordinal analyses to answer ordinal questions,” *Psychological Bulletin*, vol. 114, no. 3, pp. 494–509, 1993. [Online]. Available: <https://doi.org/10.1037/0033-2909.114.3.494>
 - [52] S. Holm, “A Simple Sequentially Rejective Multiple Test Procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979. [Online]. Available: <https://www.jstor.org/stable/4615733>
 - [53] D. Perille, A. Truong, X. Xiao, and P. Stone, “Benchmarking Metric Ground Navigation,” 2020. [Online]. Available: <https://arxiv.org/abs/2008.13315>
 - [54] X. Xiao, Z. Xu, A. Datar, G. Warnell, P. Stone, J. J. Damanik, J. Jung, C. A. Deresa, T. D. Huy, C. Jinyu, C. Yichen, J. A. Cahyono, J. Wu, L. Mo, M. Lv, B. Lan, Q. Meng, W. Tao, and L. Cheng, “Autonomous Ground Navigation in Highly Constrained Spaces: Lessons Learned from The 3rd BARN Challenge at ICRA 2024,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.01862>
 - [55] A. Nair, F. Jiang, K. Hou, Z. Xu, S. Li, X. Xiao, and P. Stone, “DynaBARN: Benchmarking Metric Ground Navigation in Dynamic Environments,” in *2022 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*. IEEE, 2022, pp. 347–352.
 - [56] N. Pérez-Higueras, R. Otero, F. Caballero, and L. Merino, “HuNavSim: A ROS 2 Human Navigation Simulator for Benchmarking Human-Aware Robot Navigation,” 2023, accepted for publication in *IEEE Robotics and Automation Letters (RA-L)*. [Online]. Available: <https://arxiv.org/abs/2305.01303>